Malicious Code Analysis : CMU Heinz College 95-759
Assignment #2 : Fundamentals

Name: Brendan Reed
Date: 27 January 2026

Exercises:

1. UPX is a very common packer used for obfuscating the code of a Windows program.
    a. If a UPX packed program is executed, how will it appear in Process Explorer when executed? What does that symbolize?

A prominent indicator that an executable may be a UPX-packed program is that it will appear PURPLE in Process Explorer. I've researched and found a few conflicting results on what exactly Process Explorer constitutes as a Packed Image. In one StackOverflow post, users said "…the exact heuristic that PE uses to detect packing is not documented. That would make it too easy to circumvent." In other sources I've found, including the Process Explorer documentation, they may determine this when strings or other portions of a packed file's in-memory code does not match the on-disk image.

Analyzing UPX packer part 1. introduction | by S chowdhury | medium. (n.d.-a). https://medium.com/@schowdhury3838/analyzing-upx-packer-part-1-df3a793a0e2e

Jimm Chen 3, Remy Lebeau – Remy Lebeau, Jimm Chen – Jimm Chen, Ryan Bemrose – Ryan Bemrose, Hans Passant 947k151151 gold badges1.8k1.8k silver badges2.6k2.6k bronze badges, Bruno Zell 8, & CSZQ 1122 bronze badges. (2016, February 17). *Windows EXE/DLL: What is a "packed image."* Stack Overflow. https://stackoverflow.com/questions/35447443/windows-exe-dll-what-is-a-packed-image

Me. (2025, July 16). *Process explorer*. Rahul Mishra. https://rivian96.github.io/blogs/process-explorer/

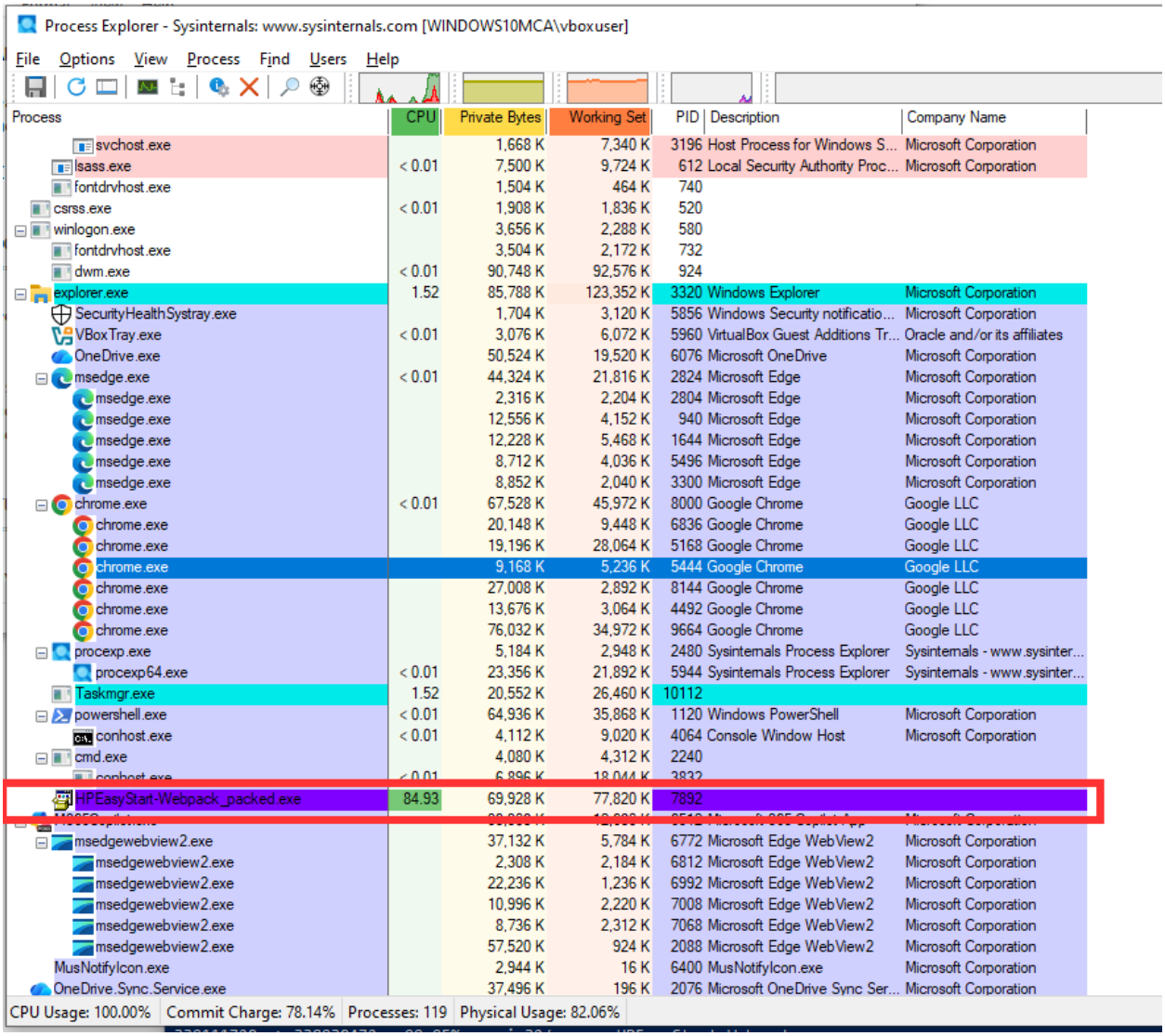
Process explorer - documentation & help. (n.d.-b). https://documentation.help/Process-Explorer/documentation.pdf

Process explorer: Part Two. ThreatDown by Malwarebytes. (2024, May 14). https://www.threatdown.com/blog/process-explorer-part-two/

UPX. (n.d.). https://upx.github.io/

What is a malware packer? detection & analysis guide. Huntress. (n.d.). https://www.huntress.com/cybersecurity-101/topic/malware-packer

b. Prove your answer by using the open source upx utility to pack an executable of your choosing, then run it on your Windows host and take a screenshot of how it looks in Process Explorer.

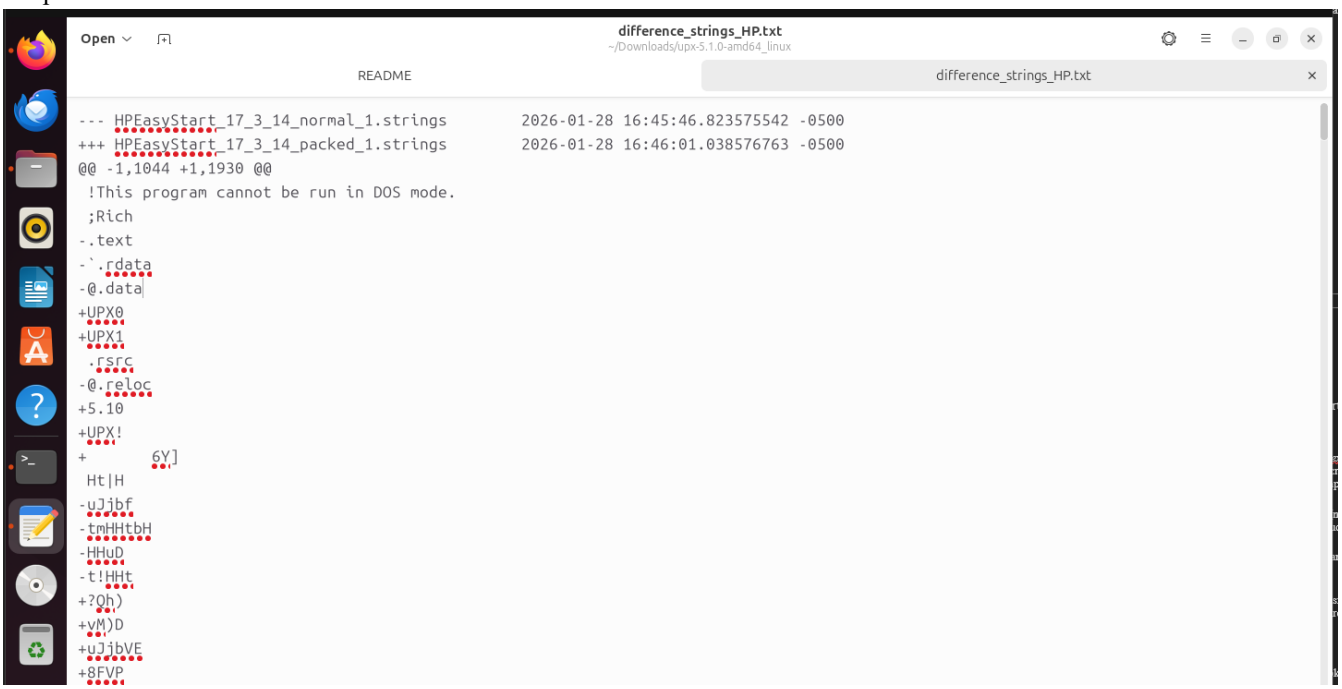


c. Compare the output of the ‘strings’ Linux utility between the two executables. What’s different between them? Why? What about this might assist malware authors?

```
user@user-ubuntu-mca:~/Downloads/upx-5.1.0-amd64_linux$ strings HPEasyStart_17_3_14_packed.exe > HPEasyStart_17_3_14_packed_1.strings
```

```
user@user-ubuntu-mca:~/Downloads/upx-5.1.0-amd64_linux$ diff -u HPEasyStart_17_3_14_normal.strings HPEasyStart_17_3_14_packed.strings > difference_strings_HP.txt
```

Output:



When UPX is used it adds readable strings indicating the packer. The following flags are present in the above strings ‘diff’ file: `UPX!, UPX0, UPX1.` This is helpful for malware analysts to determine that a file has indeed been packed. However, UPX also compresses the executable’s code and data sections including embedded strings, which obfuscates the code. This behavior assists malware authors by obscuring indicators of compromise such as URLs, file paths, registry keys, and command strings.

2. Review the PE file format and answer the following questions.
    a. What does the TimeDateStamp symbolize?

The TimeDateStamp field in the COFF Header of a Win32 Portable Executable is a 32bit value representing the EPOCH time of when the file was created.

Dotnet-Bot. (n.d.). *CoffHeader.timedatestamp property (system.reflection.portableexecutable)*. (System.Reflection.PortableExecutable) | Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/api/system.reflection.portableexecutable.coffheader.timedatestamp?view=net-10.0

File:portable executable 32 bit structure in SVG fixed.svg. Wikimedia Commons. (n.d.). https://commons.wikimedia.org/wiki/File:Portable_Executable_32_bit_Structure_in_SVG_fixed.svg

Karl-Bridge-Microsoft. (n.d.). *PE format - win32 apps*. Win32 apps | Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/debug/pe-format

Kexugit. (n.d.). *Inside windows: Win32 portable executable file format in detail*. Inside Windows: Win32 Portable Executable File Format in Detail | Microsoft Learn. https://learn.microsoft.com/en-us/archive/msdn-magazine/2002/february/inside-windows-win32-portable-executable-file-format-in-detail

    b. How many bytes inside an executable do you need to look before you locate the offset of 'TimeDateStamp' and how large is the data value in bytes (provide your answers in decimal or hexadecimal)?

The TimeDateStamp is a 32bit DWORD which translates to 4 bytes in size (0x04 in hexadecimal). In my research I've found that a standard Win32 PE file begins with the IMAGE_DOS_HEADER which is 64 bytes long. Following this is the IMAGE_DOS_STUB which is another 64 bytes. After the headers is a 4 bytes signature and 4 bytes labelled Machine & #NumberOfSections. **So, the true offset of TimeDateStamp in a normal PE file is 136 bytes which translates to 0x88 in base-16.**

Dissecting pe headers | tryhackme walkthrough | by IritT | Medium. (n.d.-a). https://iritt.medium.com/dissecting-pe-headers-tryhackme-walkthrough-cbd4c2de17da

FAUT.L blog - malware analysis & cybersecurity resources. (n.d.-b). https://blog.fautl.com/

The Malware Analyst's Guide to PE Timestamps. 0xC0DECAFE.com. (2021, January 27). https://web.archive.org/web/20210620225129/https://0xc0decafe.com/malware-analyst-guide-to-pe-timestamps/

    c. How might this data point be important for malware analysts?

The TimeDateStamp field can sometimes indicate compilation time, but this is easily forged by malware and malware authors. This is either to make the program appear more legitimate or mask true compilation time. Both the TimeDateStamp value and it's offset provide valuable information to an analyst for validating the integrity of a PE's file structure. Using these tactics, an analyst can determine if data has been malformed or tampered with. The TimeDateStamp itself can also be helpful in determine creation time and possibly malware families based on this.

d. Briefly describe the ASLR security feature and its effect on the imagebase when loaded into memory?

Address Space Layout Randomization (ASLR) is an operating system security feature to help prevent memory corruption attacks like buffer overflows. that makes exploitation and hijacking of a system harder by randomly assigning. It works by each time a program is loaded into memory, the OS randomizes the memory addresses for the different segments of the program like the executable code, the data, stack, heap, and shared libraries. This makes is more difficult to exploit a memory corruption vulnerability.

With ASLR the executable's preferred imagebase in the PE header is often ignored and replaced by the OS's randomly assigned memory location. This unpredictability prevents attackers from reliably hard-coding memory addresses for exploitation.

MalBot. (2020, March 17). *Six facts about address space layout randomization on windows*. Malware Analysis, News and Indicators. https://malware.news/t/six-facts-about-address-space-layout-randomization-on-windows/38160

Wikimedia Foundation. (2026, January 9). *Address space layout randomization*. Wikipedia. https://en.wikipedia.org/wiki/Address_space_layout_randomization

YouTube. (n.d.). YouTube. https://www.youtube.com/shorts/l_9yLUU84oA

e. How can you tell if an executable has ASLR enabled? (Hint: Look at the dllcharacteristics field).

A module that is not relocatable will have the IMAGE_FILE_RELOCS_STRIPPED (0x0001) bit flag set in the Characteristics field of the File Header. A relocatable module will have this bit cleared. This flag can be examined with software like PEView.

Shuzheng 14.7k3030 gold badges130130 silver badges252252 bronze badges, Hans Passant – Hans Passant, Shuzheng – Shuzheng, Mihai – Mihai, & Mihai 54866 silver badges1818 bronze badges. (2016, August 28). *How do I determine if an EXE (or DLL) participate in ASLR, i.e. is relocatable?*. Stack Overflow. https://stackoverflow.com/questions/39189477/how-do-i-determine-if-an-exe-or-dll-participate-in-aslr-i-e-is-relocatable